

Agent Scheduling — A Survey of the Control Layer of LLM-Agent Systems

Last updated: 2026-07-13. Scope: the control/orchestration layer of LLM agents — how an agentic system decides *what runs, when, in what topology, and with what memory state* — not general LLM pretraining.

This document expands and consolidates the seed notes in [agentMD.txt](#), the LONGAGENT deep-dive in [paper_8.md](#) / [agent_survey.md](#), and fills the gaps with the latest (2025 H2 → 2026 H1) literature.

0. Framing: three control levers, one optimizer

An LLM-agent system is a *scheduler over model calls*. Three control levers dominate the 2024–2026 literature, and a fourth theme (RL) increasingly ties them together:

Lever	Question it answers	Pillar
Policy optimization	<i>What action/tool/call should the agent choose?</i>	§1 Agent RL
Memory / context management	<i>What working state does the agent carry, and at what cost?</i>	§2 Memory optimization
Graph / orchestration	<i>Which agents run, in what topology, and in what order?</i>	§3 Call graph & dynamic arrangement
(cross-cutting) RL	<i>How is each of the above learned rather than hand-designed?</i>	§4 Synthesis

The strongest signal across the whole field is convergence: the same **verifiable-reward RL loop** now tunes tool calls (§1), memory edits (§2), and topology (§3). A 2026 survey synthesizing 500+ works ([The Landscape of Agentic RL](#)) frames this as a paradigm shift from single-step MDPs to **temporally-extended POMDPs**, where the LLM is an autonomous decision-maker rather than a one-shot responder.

1. Agent RL — policy optimization for agents

1.1 The dominant paradigm: outcome-based, verifiable-reward RL

The field has moved from step-level supervision to **end-to-end trajectory rewards with a verifiable final answer** (math correctness, code passing tests, tool-call success). The unlock — demonstrated by DeepSeek-R1 — is that RL on *binary correctness* rewards, with **GRPO** (Group Relative Policy Optimization, Shao et al. 2024) and **no reward model**, elicits sophisticated chain-of-thought. Two "waves" are now distinguished ([Credit Assignment survey, 2604.09459](#)):

- 1. **Reasoning RL** — sparse outcome rewards, ~500-token trajectories (DeepSeek-R1, o1/o3 style).
- 2. **Agentic RL** — multi-turn environment interaction, 100K–1M-token trajectories, where episode-level credit becomes uninformative and **credit assignment** is the central bottleneck.

1.2 Key systems

Paper	Core idea	Notable result
ARTIST (2505.01441)	Couples agentic reasoning + RL + tool integration; model learns <i>when/how/which</i> tool to call. Outcome-based GRPO; tool-output tokens masked from loss so the model learns <i>decisions</i> , not tool outputs.	+~22% abs on math; +8% τ -bench, +16% hardest BFCL v3; 7B beats GPT-4o. ~500 steps / 8×H100.
ARPO (2507.19849 , code)	Entropy-based adaptive rollout: token entropy spikes right after a tool call → branch Z extra partial reasoning paths when ΔH_t exceeds a threshold ($P_t = \alpha + \beta \cdot \Delta H_t$). Advantage attribution shares credit along paths.	Beats trajectory-level RL on 13 benchmarks at $\frac{1}{2}$ the tool-call budget ; rollout cost $O(n \log n)$ – $O(n^2)$ vs $O(n^2)$.

Paper	Core idea	Notable result
Agentic Entropy-Balanced PO (2510.14545)	Successor to ARPO; balances entropy across rollout and update to avoid entropy collapse.	(the "AWPO"-class entropy-balanced method)
Demystifying RL in Agentic Reasoning (2510.11701)	Systematic study of data / algorithm / reasoning-mode.	Real end-to-end trajectories > stitched synthetic SFT; fewer, deliberate tool calls beat frequent calls; 4B matches 32B with the right recipe.
Agentic Tool Use survey (2604.00835)	Dedicated survey of the tool-use capability inside agentic LLMs.	—

Practical findings that recur: real tool-use trajectories beat synthetic-stitched data; exploration tricks (clip-higher, overlong-reward shaping, sustained entropy) matter; a *deliberative* strategy with fewer tool calls outperforms frequent calling ("less is more").

1.3 Credit assignment — the current frontier

As trajectories grow from ~500 tokens to 1M+, episode-level reward is too coarse. The [Credit Assignment survey \(2604.09459\)](#) catalogs **47 methods (2024–early 2026)** and the shift toward **turn-level** and **hindsight-based** credit assignment. Curated list: [Awesome-Credit-Assignment-in-LLM-RL](#).

1.4 RLAIF / DPO context

For *alignment* rather than verifiable-score maximization: **RLAIF** (RL from AI feedback, successor to Constitutional AI) matches RLHF on summarization/dialogue with no human labels ([2309.00267](#)); a cautionary paper ([2402.12366](#)) argues the RL step may be unnecessary — **DPO often suffices**. Relevant when the reward is preference-shaped, not verifiable.

1.5 Multi-agent RL (where RL meets the graph layer)

- **EALLMs** (IEEE T-ASE 2025, [doi](#)) — one LLM as a shared online-MARL policy + one as communication integrator; beats classical MARL on SMAC/SMACv2.
 - **LLM-MARL** ([2506.04251](#)) — Coordinator / Communicator / Memory modules; PPO + language-conditioned loss; emergent role specialization.
 - **OptAgent** (Findings IJCNLP 2025, [pdf](#)) — "verbal reinforcement learning" to optimize multi-agent interaction.
 - **LLM-Coordination** (Findings NAACL 2025, [pdf](#)) — LLMs are good at environment comprehension but weak at Theory-of-Mind / joint planning.
-

2. Agent memory optimization — chunk, dynamic window, compression

The 2025 shift: from *unbounded context growth* → *bounded, learnable memory curation*. The unifying problem is **linear context growth in long-horizon agents**, attacked by three strategies.

2.1 Taxonomy (from the 2026 surveys)

- **Memory for Autonomous LLM Agents (2603.07670)** — formalizes memory as a **write–manage–read** loop; taxonomy over temporal scope × representational substrate × control policy; five mechanism families: *context-resident compression*, *retrieval-augmented stores*, *reflective self-improvement*, *hierarchical virtual context*, *policy-learned management*.
- **Context Compression for LLM Agents: A Survey (2026)** — compression **target** × **mechanism** × **control policy**; failure modes **F1 pre-compression decision error**, **F2 in-compression information loss**, **F3 post-compression access failure**. ([paper list](#))
- **From Storage to Experience (ACL 2026 Findings)** — evolutionary framework: **Storage** → **Reflection** → **Experience** (trajectory preservation → refinement → abstraction).

2.2 Strategy A — learned constant-memory state

Paper	Mechanism	Result
MemAgent (2507.20559)	RL agent with dynamically-updated fixed-length memory ; processes the stream in chunks, overwrites a fixed slot $\rightarrow$ $O(N)$ compute, never-growing window. Multi-Conv DAPO training.	8K model + 1K memory handled 4M tokens at <5% loss .
MEM1 (2506.15841 , code)	End-to-end RL for constant memory ; each turn consolidates a compact shared internal state, discarding redundant info.	MEM1-7B: 3.5$\times$ perf, 3.7$\times$ less memory vs Qwen2.5-14B on 16-objective multi-hop QA; generalizes past training horizon.

2.3 Strategy B — memory edits as RL policy actions

Paper	Mechanism	Result
MemAct / Memory-as-Action (2510.12635 , code)	Working-memory management = learnable policy actions (in-place delete/insert edits), optimized end-to-end. Trained with DCPO (Dynamic Context Policy Optimization), which fixes "trajectory fractures" from context edits via trajectory segmentation + advantage estimation.	Matches accuracy of models 16$\times$ larger while cutting context length 51% .
ACON (2510.00615 , code)	Gradient-free agent context optimization: compresses observations + history via <i>compression-guideline optimization in natural-language space</i> (failure-driven contrastive feedback); distills the compressor into small models.	–26–54% peak tokens ; distilled compressor keeps >95% acc; +up to 46% for small LMs.

2.4 Strategy C — active branching / folding of sub-trajectories

Paper	Mechanism	Result
Context-Folding (2510.11967 , site)	Agent actively branch() es into a sub-trajectory for a subtask, then folds it (collapse intermediate steps → concise outcome summary) via return() . Trained with FoldGRPO / FoldPO (dense token-level process rewards).	With 32K active budget: 62.0% BrowseComp-Plus, 58.0% SWE-Bench Verified — beats ReAct needing 327K context.
AgentFold (2510.24699)	Long-horizon web agents with proactive context management.	—
FoldAct (2512.22733)	Fixes RL non-stationarity from summary actions (gradient dilution, self-conditioning) via separated loss + full-context consistency loss.	—

2.5 Memory as a system resource

- **MemOS** ([2507.03724](#)) — a "memory OS": treats memory as a managed resource unifying **plaintext / activation / parameter** memories. Core unit **MemCube** (content + provenance/versioning metadata); components MemReader, MemScheduler, MemLifecycle, MemOperator; properties controllability / plasticity / evolvability.
- **Are We Ready for an Agent-Native Memory System?** ([2606.24775](#)) — forward-looking design critique.

Themes: (a) chunk-and-overwrite → fixed memory, linear cost, extreme length extrapolation (MemAgent, MEM1); (b) memory edits as RL actions, with DCPO/FoldGRPO overcoming the trajectory-fracture / non-stationarity problem (MemAct, Context-Folding, FoldAct); (c) natural-language compressors distilled into small models (ACON); (d) memory as an OS-level managed substrate (MemOS).

3. Agent call graph & dynamic arrangement

This is the scheduling/graph cluster: represent dependencies as graphs, and let a scheduler execute them — increasingly with the topology *chosen per query*. It splits cleanly into (3.1) **long-context agents** (the LONGAGENT lineage), (3.2) **systems-level schedulers**, (3.3) **design-time vs runtime workflow optimization**, and (3.4) **benchmarks**.

3.1 Long-context agents — the LONGAGENT lineage

LONGAGENT (EMNLP 2024) reframes long-context processing as **agent orchestration** rather than window extension: many member agents read local chunks; a leader decomposes the question, gathers evidence, resolves inter-member conflicts, and answers. (Full deep-dive in [paper_8.md](#).) Its leader is only a *primitive scheduler* — it picks among {QUERY, CONFLICT, ANSWER} and broadcasts to all members — which motivated the successors:

Work	Advancement beyond LONGAGENT
Chain-of-Agents (NeurIPS 2024, 2406.02818)	Training-free sequential workers passing a growing state → manager synthesizes. Simplest general baseline.
XpandA (ICML 2025, 2505.20625)	Dynamic partitioning (saturation function, 1K → 1M tokens) + question-guided shared memory + iterative selective replay for inverted-order evidence. First agent method beyond 128K without training; ~20% gain, 1.5× speedup.
Tree of Agents (Findings EMNLP 2025)	Multiple tree-structured reading orders vs one star; multi-perspective reasoning, adaptive pruning, prefix-hash caching. Attacks position bias.
Graph of Agents (2509.21848)	Formalizes long-context as information-theoretic compression ; builds an input-dependent collaboration graph from chunk semantics. A 2K-window GoA beats 128K Llama-3.1-8B on LongBench; +16.35% over fixed-structure MAS.

Work	Advancement beyond LONGAGENT
DocAgent (EMNLP 2025)	Multimodal: tree-formatted outline, interactive reading, reviewer agent, cross-task memory.
MDocAgent (2503.13964)	General/critical/text/image/summarization agents combining visual + textual retrieval.

Coding Agents

Are Effective Long-Context Processors (2603.20432)	Treats long context as an external environment inspected with files/retrieval/code/recursive calls — shift from "many chunk readers" to tool-using context exploration.
---	--

Trajectory: fixed chunks + leader broadcast → fixed sequential aggregation → dynamic chunks + selective replay → multiple reasoning orders → input-dependent collaboration graph → structured multimodal / tool-mediated exploration. The field is moving from **hand-specified communication patterns** toward **query-dependent routing, evidence memory, selective execution, and tool-mediated context access**.

3.2 Systems-level schedulers

System	Graph model	Scheduler	Result
DynTaskMAS (MDPI Electronics 15(11):2475, 2025)	Dynamic task graph (logical deps)	Async parallel engine + adaptive workflow mgr + semantic context mgr	−21–33% exec time, +35% resource use, near-linear to 16 agents
Gradientsys (2507.06520)	Typed MCP + ReAct planning loop	LLM scheduler: one-to-many dispatch, sync/async, capacity, retry-replan, SSE observability	> MinionS on GAIA, lower latency/cost

System	Graph model	Scheduler	Result
LLMOrch (2504.14872)	Function-call Relation	Planning	
	Graph (data def-use + control mutual-exclusion)	separated from execution; parallel across processors	2× on compute-heavy funcs; linear scaling vs ReAct/ParallelFC/LLMCompiler

3.3 Workflow optimization: design-time vs runtime

Design-time (optimize a workflow once, deploy repeatedly):

Work	Mechanism
GPTSwarm (ICML 2024, PMLR)	Agents/ops as graph nodes; optimizes prompts + graph connectivity.
ADAS (ICLR 2025, 2408.08435)	Meta Agent Search — a meta-agent writes & improves agent systems as code.
AFlow (ICLR 2025, 2410.10762)	Workflows as executable code graphs; MCTS + execution feedback.

Runtime / query-dependent (topology or agent set changes per input or execution state):

Work	Runtime-adaptive feature
DyLAN (COLM 2024, 2310.02170)	Agent Importance Score selects the team, then runs a dynamic comm network.
G-Designer (ICML 2025, PMLR v267)	Variational graph autoencoder generates a task-aware topology. −95.33% tokens on HumanEval; +0.3% robustness under attack.
MaAS (ICML 2025, PMLR v267)	Learns an agentic supernet ; samples a query-dependent architecture (LLM calls, tool calls, tokens by difficulty). 6–45% of baseline inference cost , +0.54–11.82%.

Work	Runtime-adaptive feature
Evolving Orchestration (2505.19591)	Centralized policy picks which agent to activate each step; improved via RL.
DynaSwarm (2507.23261)	Actor-critic optimizes graph structures + query-conditioned selector.
Graph-GRPO (Findings ACL 2026)	Samples groups of comm graphs per query; group-relative rewards for edge-level credit assignment .
AgentConductor (2602.17100)	RL-optimized orchestrator builds density-aware layered DAG; SFT → RL with structural+accuracy+density reward. +14.6% pass@1, -68% tokens .
Assemble Your Crew (AAAI 2026, doi)	Autoregressive graph generation for topology design.
Lemon (2605.14483)	Counterfactual RL to learn executable orchestration.
MAS-Orchestra + MASBENCH (2601.14652)	Orchestration as a function-calling RL problem — whole-graph-at-once, not step-by-step builds.
AdaptOrch (2602.16873)	Performance Convergence Scaling Law (topology beats model once model perf converges) + $O(V + E)$ topology router + adaptive synthesis with termination guarantees. +12–23% over static single-topology.

3.4 Benchmarks that measure scheduling quality (topology vs cost/latency)

- **MultiAgentBench / MARBLE** (ACL 2025, [pdf](#), [code](#)) — compares **star / chain / tree / graph-mesh**; **graph best**; group discussion worst; more agents → lower KPI past a point. Token usage scaled per protocol (Fig. 5).
- **MAFBench** ([repo](#)) — isolates framework-level effects (orchestration, memory, planning, specialization, coordination) from model capability; cost-aware backend routing; three paradigms (graph-based, role-based, environment/simulation-mediated).
- **MASBENCH** (with MAS-Orchestra, [2601.14652](#)) — *first controlled* benchmark for **when/why MAS beats single-agent**, over five axes: **Depth, Horizon, Breadth,**

Parallel, Robustness. Finding: MAS gains depend on task structure + verification + orchestrator capability, not universally.

- **AMAS** (EMNLP 2025 Industry, [pdf](#)) — adaptively selects communication topology.
 - **AgentBalance** ([2512.11426](#)) — backbone-then-topology design under budget; explicitly trades cost vs performance.
-

4. Cross-cutting synthesis

1. **RL is the unifying optimizer across all three layers.** Agentic RL tunes tool calls (ARPO/ARTIST), memory edits (MemAct/DCPO, Context-Folding/FoldGRPO), and topology (Graph-GRPO, AgentConductor, Lemon, MAS-Orchestra). The same verifiable-reward loop recurs everywhere; **credit assignment over long trajectories** is the shared open problem ([2604.09459](#)).
2. **Bounded memory + graph scheduling compound.** MemAgent/MEM1-style fixed memory makes long-horizon agent graphs (DynTaskMAS/LLMOrch) cheap to run; together they enable the "millions of tokens, linear cost, parallel agents" regime.
3. **Topology is a first-class scheduling decision, not a given.** Benchmarks now show graph/scale-free > star/hierarchical on cost; over-provisioning agents *or* tokens hurts. **AdaptOrch's convergence scaling law** makes the strong claim that once model performance converges across providers, *topology* dominates system performance — pushing dynamic topology selection (G-Designer, MaAS, AgentConductor) to center stage.
4. **"Less is more" recurs.** Fewer tool calls (Demystifying), smaller groups (MultiAgentBench), fixed memory (MemAgent/MEM1), sparser topology (G-Designer -95% tokens) all beat "more."
5. **The convergence point — a budget-aware evidence scheduler.** The clearest open opportunity (developed in [paper_8.md](#)) is to replace LONGAGENT's handcrafted leader with a learned scheduler whose state is $\mathbf{s}_t = (\mathbf{q}, \text{memory } \mathbf{M}_t, \text{evidence } \mathbf{E}_t, \text{unresolved } \mathbf{U}_t, \text{budget } \mathbf{B}_t)$ and whose action is $\mathbf{a}_t = (\text{agents } \mathcal{A}_t, \text{chunks/tools } \mathcal{C}_t, \text{topology } \mathbf{G}_t, \text{op } \mathbf{r}_t, \text{stop } \mathbf{z}_t)$, optimizing:

$$\max_{\pi} \mathbb{E} [Q_{\text{answer}} + \alpha \cdot Q_{\text{grounding}} + \beta \cdot Q_{\text{coverage}} \\ - \lambda_1 \cdot C_{\text{tokens}} - \lambda_2 \cdot C_{\text{latency}} - \lambda_3 \cdot R_{\text{hallucination}}]$$

This single objective unifies XpandA adaptive partitioning + G-Designer/GoA topology generation + MaAS resource allocation + LONGAGENT conflict resolution + DocAgent verification + Graph-GRPO credit assignment.

5. Suggested reading order

Long-context agents: LONGAGENT → Chain-of-Agents → XpandA → Tree of Agents → Graph of Agents.

Auto-orchestration: GPTSwarm → ADAS / AFlow → G-Designer → MaAS → Graph-GRPO → AgentConductor / Lemon.

Agentic RL: DeepSeek-R1 (GRPO) → ARTIST → ARPO → Demystifying RL in Agentic Reasoning → Credit-Assignment survey (2604.09459).

Memory: MemAgent → MEM1 → MemAct/DCPO → Context-Folding → ACON → MemOS; then the two 2026 surveys (2603.07670, Context-Compression survey).

Surveys first (if starting cold): [Landscape of Agentic RL \(2509.02547\)](#) for the whole field; [Memory for Autonomous LLM Agents \(2603.07670\)](#) for memory; MASBENCH/MAFBench for orchestration evaluation.

6. Sources

Agent RL

- Landscape of Agentic RL (survey) — <https://arxiv.org/html/2509.02547>
- Credit Assignment in RL for LLMs (survey) — <https://www.arxiv.org/pdf/2604.09459>
- Agentic Tool Use (survey) — <https://arxiv.org/pdf/2604.00835>
- RL for Large Reasoning Models (survey) — <https://arxiv.org/pdf/2509.08827>
- ARTIST — <http://arxiv.org/pdf/2505.01441>

- ARPO — <https://arxiv.org/abs/2507.19849> · code <https://github.com/RUC-NLPIR/ARPO>
- Agentic Entropy-Balanced PO — <https://arxiv.org/html/2510.14545>
- Demystifying RL in Agentic Reasoning — <https://arxiv.org/pdf/2510.11701>
- RLAIIF vs RLHF — <https://arxiv.org/html/2309.00267v3>
- Critical eval of AI feedback — <https://arxiv.org/pdf/2402.12366>
- EALLMs — <https://doi.org/10.1109/tase.2025.3637862>
- LLM-MARL — <https://arxiv.org/pdf/2506.04251>
- OptAgent — <https://aclanthology.org/2025.findings-ijcnlp.107.pdf>
- LLM-Coordination — <https://aclanthology.org/2025.findings-naacl.448.pdf>

Memory optimization

- Memory for Autonomous LLM Agents (survey) — <https://arxiv.org/pdf/2603.07670>
- Context Compression for LLM Agents (survey) — <https://doi.org/10.20944/preprints202605.2065.v1> · list <https://github.com/YerbaPage/Awesome-Context-Compression>
- From Storage to Experience (survey) — <https://aclanthology.org/2026.findings-acl.2069/>
- MemAgent — <https://arxiv.org/abs/2507.20559>
- MEM1 — <https://arxiv.org/abs/2506.15841> · code <https://github.com/MIT-MI/MEM1>
- MemAct / DCPO — <https://arxiv.org/abs/2510.12635> · code <https://github.com/ADaM-BJTU/MemAct>
- ACON — <https://arxiv.org/abs/2510.00615> · code <https://github.com/microsoft/acon>
- Context-Folding — <https://arxiv.org/abs/2510.11967> · <https://context-folding.github.io>
- AgentFold — <https://arxiv.org/abs/2510.24699>
- FoldAct — <https://arxiv.org/abs/2512.22733>
- MemOS — <https://arxiv.org/html/2507.03724v4/>
- Agent-Native Memory — <https://arxiv.org/html/2606.24775v1>

Call graph / orchestration

- LONGAGENT — <https://aclanthology.org/2024.emnlp-main.912/>
- Chain-of-Agents — <https://arxiv.org/abs/2406.02818>
- XpandA — <https://arxiv.org/html/2505.20625v1>
- Graph of Agents — <https://arxiv.org/html/2509.21848v1>
- MDocAgent — <https://arxiv.org/abs/2503.13964>

- Coding Agents as Long-Context Processors — <https://arxiv.org/html/2603.20432v1>
- DynTaskMAS (MDPI Electronics 15(11):2475) — dynamic task graph
- Gradientsys — <https://arxiv.org/abs/2507.06520>
- LLMOrch — <https://arxiv.org/html/2504.14872>
- GPTSwarm — <https://proceedings.mlr.press/v235/zhuge24a.html>
- ADAS — <https://arxiv.org/abs/2408.08435>
- AFlow — <https://arxiv.org/abs/2410.10762>
- DyLAN — <https://arxiv.org/abs/2310.02170>
- G-Designer — <https://proceedings.mlr.press/v267/zhang25cu.html>
- MaAS — <https://proceedings.mlr.press/v267/zhang25bi.html>
- Evolving Orchestration — <https://arxiv.org/html/2505.19591v1>
- DynaSwarm — <https://arxiv.org/abs/2507.23261>
- AgentConductor — <https://www.arxiv.org/pdf/2602.17100>
- Assemble Your Crew (AAAI 2026) — <https://doi.org/10.1609/aaai.v40i28.39481>
- Lemon (counterfactual RL orchestration) — <https://arxiv.org/html/2605.14483v1>
- MAS-Orchestra + MASBENCH — <https://arxiv.org/abs/2601.14652>
- AdaptOrch — <https://arxiv.org/abs/2602.16873>

Benchmarks

- MultiAgentBench / MARBLE — <https://aclanthology.org/2025.acl-long.421/> · code <https://github.com/MultiagentBench/MARBLE>
- MAFBench — <https://github.com/CoDS-GCS/MAFBench>
- AMAS — <https://aclanthology.org/anthology-files/pdf/emnlp/2025.emnlp-industry.144.pdf>
- AgentBalance — <https://arxiv.org/pdf/2512.11426>

Note on provenance: papers dated 2601–2606 are 2026 (H1) arXiv preprints surfaced via web search; a few identifiers were only web-verified, not fetched in full. Where the seed notes and web results disagreed on an arXiv ID (e.g. MemAgent 2507.20559 vs 2507.2259), the well-formed ID from the code/paper listing was used. Cross-check IDs before citing in a formal venue.